Project 1: Container

Release Date: Thursday, Feburary 19, 2026
Due Date: Thursday, March 5, 2026, 11:59 PM EST

Overview

The goal of this project is to familiarize you with the lifecycle of cloud-native applications. You are provided with a legacy Python script. Your objective is to modernize this application by **containerizing** it and **orchestrating** it on a Kubernetes cluster.

You have the flexibility to choose your infrastructure. You may complete this project using a **Local Environment** (simulated cluster) or a **Public Cloud Platform**.

> ⚠️ **Cost Warning for Cloud Users:** If you choose to use a public cloud provider, you are responsible for monitoring your own billing. Managed Kubernetes services (like EKS or GKE) often fall outside of “Free Tier” limits and may incur costs ($0.10/hour + resources). **Remember to shut down your cluster immediately after finishing.**

Part A: The Application Source

Create a file named `app.py` with the following content. This is the application you will be working with.

```
from flask import Flask
import os
import socket

app = Flask(__name__)

@app.route("/")
def hello():
    html = "<h3>Hello, Cloud!</h3>" \
           "<b>Hostname:</b> {hostname}<br/>"
    return html.format(hostname=socket.gethostname())

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

Part B: Project Tasks

Complete the following tasks. As you work, keep a detailed log of your actions. You will need to submit a report containing the **commands you ran**, the **output/results** generated, and **explanations** of your approach.

Task 1: Containerization

Create a `Dockerfile` to package the `app.py` application.

- **Goal:** The container must allow the application to run successfully and listen on the correct port.
- **Constraint:** You must choose an appropriate base image and handle the installation of necessary dependencies (e.g., `flask`).
- **Validation:** Build the image locally and run a container instance (using pure Docker commands) to verify that you can access the webpage from your browser.

Task 2: Cluster Provisioning

Chat Assistant

Set up your Kubernetes environment.

- **Goal:** Provision a operational Kubernetes cluster.
  - *Local Users:* Install and start your chosen tool (e..g, Minikube/Kind).
  - *Cloud Users:* Create a managed cluster (e.g., EKS/GKE) or manually install Kubernetes on Virtual Machines.
- **Validation:** Run a command to confirm the cluster is active, the nodes are `Ready`, and your `kubectl` CLI is configured to talk to the cluster.

Task 3: Application Deployment

Deploy your containerized application to the cluster.

- **Goal:** The application should run as a **Deployment** with multiple replicas (at least 2) for high availability.
- **The "Image Access" Challenge:**
  - *Local Users:* You must figure out how to make your locally built image accessible to the cluster (e.g., loading it into the cluster cache) without Docker Hub.
  - *Cloud Users:* Your remote cluster cannot see images on your laptop. You must figure out how to push your image to a registry (Docker Hub, ECR, GCR) and configure your deployment to pull from there.
- **Validation:** Verify that the pods are running and **not** stuck in `ImagePullBackOff` or `ErrImagePull`.

Task 4: Service Exposure

Expose your application so it can be accessed from the internet (or your local browser).

- **Goal:** Create a Kubernetes **Service** to route traffic to your pods.
- **Research Required:** Choose the Service type best suited for your platform:
  - *Local Users:* `NodePort` or `ClusterIP` (with port-forwarding).
  - *Cloud Users:* `LoadBalancer` is the standard approach, though `NodePort` (with correct Firewall/Security Group rules) is acceptable.
- **Validation:** Access the application URL. Refresh the page multiple times and observe the "Hostname" value to verify traffic is hitting different pods.

Task 5: Lifecycle Management (Scaling & Updating)

Demonstrate the elasticity and update capabilities of Kubernetes.

1. **Scaling:** Dynamically increase the number of replicas

2. **Rolling Update:** Modify the `app.py` source code (e.g., change the text to "Hello, Winter 2026"). Build a version 2 image, and perform a rolling update of your deployment with zero downtime.

Task 6: Teardown

Clean up your environment.

- **Goal:** Remove all application resources (Services, Deployments).
- **Critical Step:** Stop or delete the cluster to ensure no further resources are being consumed (especially important for Cloud users to avoid billing).

Submission Guidelines

Due Date: Thursday, March 5, 11:59 PM.

Format: Submit a **SINGLE PDF** file named `Project1_YourName.pdf` to Brightspace.

Report Requirements: Your report is the primary evidence of your work. For **EACH** task listed above, you must include:

1. **Commands Run:** The exact CLI commands you executed.
2. **Output:** The text output from those commands (e.g., `pod/my-app created`, `STATUS: Running`) or screenshots where applicable.
3. **Source Code:** Copy and paste the full content of any configuration files you created (`Dockerfile`, `deployment.yaml`, `service.yaml`) directly into the PDF. **Do not upload separate code files.**
4. **Explanation:** A brief paragraph explaining *why* you ran those commands or configured the YAML files that way.

Grading Scheme

The project is graded out of **100 points**.

Category	Points	Criteria
Documentation & Evidence	**30**	**Did you show your work?** Full points if the report includes the commands run and their output/screenshots for every task. Deductions for missing logs or vague descrip

Chat Assistant

Category	Points	Criteria
Technical Correctness	**30**	**Does the code look right?** Full points for a correct `Dockerfile`, valid YAML syntax, and correct port mappings. Deductions for hardcoding IP addresses where dynamic names should be used, or basic syntax errors.
Concept Mastery	**20**	**Do you understand what you did?** Points awarded for the **Explanations** in your report. • *Excellent (20):* Clear reasoning for your choice of Platform, Service type, and Image strategy. • *Average (15):* Generic explanations. • *Poor (0-10):* No explanation, or fundamental misunderstandings (e.g., confusing the container port with the service port).
Execution	**20**	**Did it actually work?** Evidence (screenshots) showing the app running in the browser, scaling successfully, and updating the message after the rolling update.

Common Deductions:

- **-10 points:** Submitting separate code files instead of including them in the PDF.
- **-10 points:** Missing the “Hostname” observation in Task 4 (which proves load balancing is working).
- **-15 points:** Pods shown in error states (e.g., `CrashLoopBackOff` or `ImagePullBackOff`) in the final screenshots.

Chat Assistant